



ПРОГРАММИРОВАНИЕ НА PYTHON

Модуль 4: Специализация

Веб • Данные • Telegram-боты • GUI • Регулярные выражения

Глава 23: Регулярные выражения (re)

23.1 Что такое регулярные выражения?

Регулярные выражения (regex) — это мощный мини-язык для поиска и замены текста по шаблону. Умение работать с regex — один из главных навыков разработчика.

Шаблон	Что означает
.	Любой символ (кроме \n)
\d	Цифра (0–9)
\w	Буква, цифра или _
\s	Пробел, таб, перенос строки
[abc]	Один из символов: a, b или c
[a-z]	Любая строчная латинская буква
^	Начало строки
\$	Конец строки
+	Один или более повторов
*	Ноль или более повторов
?	Ноль или один раз (необязательный)
{3}	Ровно 3 повтора
{2,5}	От 2 до 5 повторов
(abc)	Группа захвата

23.2 Основные функции модуля re

```
import re

text = 'Позвони мне: +7 (495) 123-45-67 или +7 (812) 987-65-43'

# re.findall — найти все совпадения
phones = re.findall(r'\+7 \(\d{3}\) \d{3}-\d{2}-\d{2}', text)
print(phones)    # ['+7 (495) 123-45-67', '+7 (812) 987-65-43']

# re.search — найти первое совпадение
m = re.search(r'\d{3}-\d{2}-\d{2}', text)
if m:
    print(m.group())    # 123-45-67
    print(m.start())    # позиция начала

# re.sub — замена
clean = re.sub(r'\+7 \(\d{3}\) ', '***-', text)
print(clean)    # Позвони мне: ***-123-45-67 или ***-987-65-43

# re.split — разбить по шаблону
parts = re.split(r'[,\s]+', 'яблоко, банан; груша слива')
print(parts)    # ['яблоко', 'банан', 'груша', 'слива']
```

23.3 Практические примеры

```
import re

def validate_email(email):
    pattern = r'^[\w.+~]+@[\w-]+\.[a-zA-Z]{2,}$'
    return bool(re.match(pattern, email))

def extract_urls(text):
    return re.findall(r'https?://[^\s]+', text)

def mask_card(text):
    # Маскируем номера карт: 4111 1111 1111 1111 -> **** * 1111
    return re.sub(r'\b(\d{4}) {3}', '**** ' * 3, text)

print(validate_email('user@example.com')) # True
print(validate_email('not-an-email'))     # False

news = 'Подробнее на https://python.org и https://docs.python.org/3'
print(extract_urls(news)) # ['https://python.org',
                           'https://docs.python.org/3']
```

 **Задание 1:** Напиши функцию `parse_log(text)`, которая находит в тексте все строки формата '[ERROR] сообщение' и возвращает список только сообщений об ошибках.

Глава 24: Веб-скрапинг (BeautifulSoup)

24.1 Что такое веб-скрапинг?

Веб-скрапинг — автоматическое извлечение данных с веб-страниц. Используется для сбора данных, мониторинга цен, парсинга новостей.

⚠ Этика скрапинга: Всегда проверяй файл robots.txt сайта. Не перегружай сервер запросами. Используй паузы между запросами. Не нарушай авторские права.

```
# Установка:
# pip install beautifulsoup4 requests

import requests
from bs4 import BeautifulSoup
import time

def scrape_quotes(url):
    headers = {'User-Agent': 'Mozilla/5.0'}
    response = requests.get(url, headers=headers, timeout=10)
    response.raise_for_status()

    soup = BeautifulSoup(response.text, 'html.parser')

    quotes = []
    for block in soup.find_all('div', class_='quote'):
        text = block.find('span', class_='text').get_text()
        author = block.find('small', class_='author').get_text()
        tags = [t.get_text() for t in block.find_all('a', class_='tag')]
        quotes.append({'text': text, 'author': author, 'tags': tags})

    return quotes

quotes = scrape_quotes('http://quotes.toscrape.com')
for q in quotes[:3]:
    print(f'{q["author"]}: {q["text"][:60]}...')
    time.sleep(0.5) # пауза между запросами
```

24.2 Навигация по HTML

Метод	Что делает
<code>soup.find('div')</code>	Первый тег div
<code>soup.find_all('a')</code>	Все теги a
<code>soup.find('div', id='x')</code>	Div с id='x'
<code>soup.find('p', class_='c')</code>	Абзац с классом c
<code>tag.get_text()</code>	Текст внутри тега
<code>tag['href']</code>	Значение атрибута href
<code>tag.find_parent('div')</code>	Ближайший родитель div
<code>tag.find_next_sibling()</code>	Следующий сосед

24.3 Сохранение данных в CSV

```
import csv
import requests
from bs4 import BeautifulSoup

def save_to_csv(data, filename):
    if not data: return
    with open(filename, 'w', newline='', encoding='utf-8') as f:
        writer = csv.DictWriter(f, fieldnames=data[0].keys())
        writer.writeheader()
        writer.writerows(data)
    print(f'Сохранено {len(data)} записей в {filename}')

quotes = scrape_quotes('http://quotes.toscrape.com')
# tags — список, преобразуем в строку для CSV
for q in quotes:
    q['tags'] = ', '.join(q['tags'])
save_to_csv(quotes, 'quotes.csv')
```

 **Задание 2:** Напиши скрапер для <http://books.toscrape.com> — извлеки название, цену и рейтинг первых 20 книг. Сохрани в CSV. Сайт специально создан для практики скрапинга.

Глава 25: Веб-разработка: Flask

25.1 Что такое Flask?

Flask — лёгкий веб-фреймворк. Позволяет создавать веб-сайты и API на Python буквально за несколько минут. Идеален для обучения и небольших проектов.

```
# pip install flask

from flask import Flask, render_template, request, jsonify

app = Flask(__name__)

# Простой маршрут
@app.route('/')
def index():
    return '<h1>Привет из Flask!</h1>'

# Маршрут с параметром
@app.route('/hello/<name>')
def hello(name):
    return f'<h2>Привет, {name}!</h2>'

if __name__ == '__main__':
    app.run(debug=True) # запустить на localhost:5000
```

❑ **Как запустить:** Сохрани как app.py, запусти python app.py. Открой браузер: <http://localhost:5000> — увидишь свой сайт!

25.2 Шаблоны HTML (Jinja2)

```
# app.py
from flask import Flask, render_template
app = Flask(__name__)

students = [
    {'name': 'Алёша', 'grade': '9А', 'score': 95},
    {'name': 'Маша', 'grade': '9Б', 'score': 88},
]

@app.route('/students')
def show_students():
    return render_template('students.html', students=students)
```

```
<!-- templates/students.html -->
<!DOCTYPE html>
<html>
<head><title>Список учеников</title></head>
<body>
  <h1>Ученики</h1>
  <table border='1'>
    <tr><th>Имя</th><th>Класс</th><th>Баллы</th></tr>
    {% for s in students %}
    <tr>
      <td>{{ s.name }}</td>
      <td>{{ s.grade }}</td>
      <td>{{ s.score }}</td>
    </tr>
    {% endfor %}
  </table>
</body>
</html>
```

25.3 REST API с Flask

```
from flask import Flask, jsonify, request
app = Flask(__name__)

tasks = [
    {'id': 1, 'title': 'Выучить Flask', 'done': False},
    {'id': 2, 'title': 'Сделать проект', 'done': True},
]

@app.route('/api/tasks', methods=['GET'])
def get_tasks():
    return jsonify(tasks)

@app.route('/api/tasks', methods=['POST'])
def add_task():
    data = request.get_json()
    new_task = {'id': len(tasks)+1, 'title': data['title'], 'done': False}
    tasks.append(new_task)
    return jsonify(new_task), 201

@app.route('/api/tasks/<int:task_id>', methods=['DELETE'])
def delete_task(task_id):
    global tasks
    tasks = [t for t in tasks if t['id'] != task_id]
    return jsonify({'status': 'deleted'})
```

 **Задание 3:** Создай Flask-приложение: простой калькулятор ИМТ (индекс массы тела). HTML-форма принимает рост и вес, Python считает ИМТ, страница показывает результат с оценкой (недовес/норма/избыток).

Глава 26: Анализ данных: Pandas

26.1 Что такое Pandas?

Pandas — главная библиотека Python для анализа данных. DataFrame — это таблица данных, с которой можно делать всё: фильтровать, сортировать, считать статистику, объединять.

```
# pip install pandas
import pandas as pd

# Создание DataFrame
data = {
    'Имя':      ['Алёша', 'Маша', 'Дима', 'Катя', 'Вова'],
    'Класс':    ['9А', '9Б', '9А', '9Б', '9А'],
    'Оценка':   [95, 88, 72, 91, 65],
    'Город':    ['Москва', 'СПб', 'Москва', 'Казань', 'СПб'],
}
df = pd.DataFrame(data)
print(df)
print(df.shape)      # (5, 4) — строки, столбцы
print(df.dtypes)     # типы данных
print(df.describe()) # статистика числовых столбцов
```

26.2 Фильтрация и выборка

```
# Выбор столбца
print(df['Оценка'])      # Series
print(df[['Имя', 'Оценку']]) # DataFrame

# Фильтрация строк
high = df[df['Оценка'] >= 90]
print(high)

# Несколько условий
result = df[(df['Класс'] == '9А') & (df['Оценка'] > 70)]

# Сортировка
sorted_df = df.sort_values('Оценка', ascending=False)

# Новый столбец
df['Статус'] = df['Оценка'].apply(lambda x: 'Отлично' if x>=90 else
                                   'Хорошо' if x>=75 else 'Удовл.')
print(df)
```


26.3 Группировка и агрегация

```
# Среднее по классам
by_class = df.groupby('Класс')['Оценка'].mean()
print(by_class)

# Несколько агрегаций сразу
stats = df.groupby('Класс')['Оценка'].agg(['mean', 'min', 'max', 'count'])
print(stats)

# Сводная таблица
pivot = df.pivot_table(values='Оценка', index='Класс', columns='Город',
                        aggfunc='mean', fill_value=0)
print(pivot)
```

```
Класс
9А      77.3
9Б      89.5
Name: Оценка, dtype: float64
```

26.4 Загрузка и сохранение данных

```
# CSV
df = pd.read_csv('data.csv', encoding='utf-8')
df.to_csv('output.csv', index=False)

# Excel
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
df.to_excel('output.xlsx', index=False)

# JSON
df = pd.read_json('data.json')
df.to_json('output.json', orient='records', force_ascii=False)

# Работа с пропущенными значениями
print(df.isnull().sum())          # сколько NaN в каждом столбце
df = df.dropna()                  # удалить строки с NaN
df = df.fillna(0)                 # заменить NaN на 0
df['col'] = df['col'].fillna(df['col'].mean()) # заменить на среднее
```

 **Задание 4:** Скачай любой CSV с данными (например с kaggle.com или data.gov.ru). Загрузи в Pandas: найди топ-10 записей, посчитай среднее/макс/мин по числовому столбцу, отфильтруй по условию и сохрани результат в новый CSV.

Глава 27: Визуализация данных: Matplotlib

27.1 Основы Matplotlib

```
# pip install matplotlib
import matplotlib.pyplot as plt
import numpy as np

# Линейный график
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 6, 3]

plt.figure(figsize=(8, 4))      # размер холста
plt.plot(x, y, marker='o', linewidth=2, color='steelblue', label='Данные')
plt.title('Мой первый график', fontsize=14)
plt.xlabel('Ось X')
plt.ylabel('Ось Y')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('graph.png', dpi=150) # сохранить в файл
plt.show()
```

27.2 Типы графиков

Тип	Функция	Когда использовать
Линейный	plt.plot()	Динамика во времени
Столбчатый	plt.bar()	Сравнение категорий
Горизонтальный	plt.barh()	Длинные подписи
Круговая диаграмма	plt.pie()	Доли целого
Точечный	plt.scatter()	Связь двух переменных
Гистограмма	plt.hist()	Распределение значений
Тепловая карта	plt.imshow()	Матрицы, корреляции

27.3 Несколько графиков на одном холсте

```
import matplotlib.pyplot as plt
import numpy as np

months = ['Янв', 'Фев', 'Мар', 'Апр', 'Май', 'Июн']
sales_a = [45, 52, 48, 61, 70, 66]
sales_b = [30, 38, 42, 55, 48, 60]

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Линейный
axes[0].plot(months, sales_a, 'o-', label='Продукт A', color='steelblue')
axes[0].plot(months, sales_b, 's--', label='Продукт B', color='tomato')
axes[0].set_title('Продажи по месяцам')
axes[0].legend()
axes[0].grid(alpha=0.3)

# Столбчатый
x = np.arange(len(months))
axes[1].bar(x - 0.2, sales_a, 0.4, label='A', color='steelblue')
axes[1].bar(x + 0.2, sales_b, 0.4, label='B', color='tomato')
axes[1].set_xticks(x)
axes[1].set_xticklabels(months)
axes[1].set_title('Сравнение продуктов')
axes[1].legend()

plt.tight_layout()
plt.savefig('sales.png', dpi=150)
```

27.4 Pandas + Matplotlib

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('students.csv')

# Pandas умеет строить графики напрямую!
df['Оценка'].hist(bins=10, color='steelblue', edgecolor='white')
plt.title('Распределение оценок')
plt.xlabel('Оценка')
plt.ylabel('Количество учеников')
plt.savefig('grades.png')

# Средняя оценка по классам — столбчатый
df.groupby('Класс')['Оценка'].mean().plot(kind='bar', color='coral')
plt.title('Средний балл по классам')
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig('by_class.png')
```

 **Задание 5:** Возьми данные из задания 4 (CSV). Построй 3 графика: 1) гистограмму числового столбца, 2) топ-10 записей (столбчатый), 3) динамику если есть временной ряд. Сохрани все в PNG.

Глава 28: Telegram-боты (aiogram)

28.1 Создание бота

Telegram-бот — программа, которая общается с пользователями через Telegram. Для создания нужен токен от @BotFather.

Шаги для создания бота:

1. Открой Telegram, найди @BotFather
2. Отправь команду /newbot
3. Придумай имя и username (должен оканчиваться на 'bot')
4. Получи TOKEN — сохрани его, он понадобится в коде

⚠️ Безопасность: Никогда не публикуй токен в открытом коде! Храни в переменной окружения или файле .env, добавь .env в .gitignore.

```
# pip install aiogram python-dotenv

import asyncio
from aiogram import Bot, Dispatcher, types
from aiogram.filters import Command

TOKEN = 'ВАШ_ТОКЕН_ЗДЕСЬ'    # лучше брать из .env!

bot = Bot(token=TOKEN)
dp = Dispatcher()

@dp.message(Command('start'))
async def start(message: types.Message):
    name = message.from_user.first_name
    await message.answer(
        f'Привет, {name}! Я тестовый бот.\n'
        f'Отправь мне любое сообщение!'
    )

@dp.message(Command('help'))
async def help_cmd(message: types.Message):
    await message.answer(
        '/start — начало\n'
        '/help — помощь\n'
        '/calc — калькулятор'
    )

@dp.message()
async def echo(message: types.Message):
    await message.answer(f'Ты написал: {message.text}')

async def main():
    await dp.start_polling(bot)

if __name__ == '__main__':
    asyncio.run(main())
```

28.2 Кнопки и клавиатуры

```
from aiogram import Bot, Dispatcher, types, F
from aiogram.filters import Command
from aiogram.utils.keyboard import ReplyKeyboardBuilder,
InlineKeyboardBuilder

@dp.message(Command('menu'))
async def menu(message: types.Message):
    builder = ReplyKeyboardBuilder()
    builder.button(text='Погода')
    builder.button(text='Курс валют')
    builder.button(text='Шутка')
    builder.adjust(2) # 2 кнопки в ряд
    await message.answer(
        'Выбери раздел:',
        reply_markup=builder.as_markup(resize_keyboard=True)
    )

@dp.message(F.text == 'Шутка')
async def joke(message: types.Message):
    import random
    jokes = [
        'Программист пишет код — баг. Удаляет код — баг.',
        'Есть 10 видов людей: те, кто понимает двоичный код, и те, кто
нет.',
    ]
    await message.answer(random.choice(jokes))
```

28.3 Состояния (FSM)

Машина состояний (Finite State Machine) позволяет боту вести диалог — запоминать, на каком шаге находится пользователь.

```
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup

class RegistrationForm(StatesGroup):
    name = State()
    age = State()
    city = State()

@dp.message(Command('register'))
async def start_reg(message: types.Message, state: FSMContext):
    await state.set_state(RegistrationForm.name)
    await message.answer('Как тебя зовут?')

@dp.message(RegistrationForm.name)
async def process_name(message: types.Message, state: FSMContext):
    await state.update_data(name=message.text)
    await state.set_state(RegistrationForm.age)
    await message.answer(f'Привет, {message.text}! Сколько тебе лет?')

@dp.message(RegistrationForm.age)
async def process_age(message: types.Message, state: FSMContext):
    await state.update_data(age=message.text)
    await state.set_state(RegistrationForm.city)
    await message.answer('Из какого ты города?')

@dp.message(RegistrationForm.city)
async def process_city(message: types.Message, state: FSMContext):
    data = await state.get_data()
    await state.clear()
    await message.answer(
        f'Отлично! Вот что я узнал:\n'
        f'Имя: {data["name"]}\n'
        f'Возраст: {data["age"]}\n'
        f'Город: {message.text}'
    )
```

 **Задание 6:** Создай Telegram-бота «Викторина»: бот задаёт 5 вопросов по Python (с вариантами ответов через inline-кнопки), считает правильные ответы и в конце показывает результат.

Глава 29: GUI-приложения (Tkinter)

29.1 Первое окно

Tkinter — встроенная библиотека для создания графических приложений. Не нужно ничего устанавливать — она есть в стандартном Python.

```
import tkinter as tk
from tkinter import ttk, messagebox

# Создаём главное окно
root = tk.Tk()
root.title('Моё первое приложение')
root.geometry('400x300') # ширина x высота
root.resizable(False, False)

# Виджеты
label = tk.Label(root, text='Введи своё имя:', font=('Arial', 12))
entry = tk.Entry(root, font=('Arial', 12), width=25)
button = tk.Button(
    root, text='Поприветствовать',
    font=('Arial', 11), bg='steelblue', fg='white',
    command=lambda: messagebox.showinfo('Привет', f'Привет, {entry.get()}!')
)
result = tk.Label(root, text='', font=('Arial', 13), fg='green')

# Расположение
label.pack(pady=(40, 5))
entry.pack(pady=5)
button.pack(pady=15)
result.pack()

root.mainloop() # запуск главного цикла
```

29.2 Основные виджеты

Виджет	Описание
<code>tk.Label</code>	Текст или картинка
<code>tk.Entry</code>	Однострочное поле ввода
<code>tk.Text</code>	Многострочное поле
<code>tk.Button</code>	Кнопка
<code>tk.Checkbutton</code>	Чекбокс (флажок)
<code>tk.Radiobutton</code>	Переключатель
<code>tk.Listbox</code>	Список с прокруткой
<code>tk.Combobox (ttk)</code>	Выпадающий список
<code>tk.Scale</code>	Ползунок
<code>tk.Frame</code>	Контейнер для группировки
<code>ttk.Treeview</code>	Таблица / дерево

29.3 Полноценное приложение — Калькулятор

```
import tkinter as tk

class Calculator:
    def __init__(self, root):
        self.root = root
        root.title('Калькулятор')
        root.geometry('320x450')
        root.configure(bg='#1e1e1e')
        self.expr = ''
        self._build_ui()

    def _build_ui(self):
        # Дисплей
        self.display = tk.Label(
            self.root, text='0', font=('Arial', 28), anchor='e',
            bg='#1e1e1e', fg='white', padx=15, pady=10)
        self.display.pack(fill='x')

        # Кнопки
        buttons = [
            ['C', '+/-', '%', '/'],
            ['7', '8', '9', '*'],
            ['4', '5', '6', '-'],
            ['1', '2', '3', '+'],
            ['0', '.', '⊞', '='],
        ]
        frame = tk.Frame(self.root, bg='#1e1e1e')
        frame.pack(fill='both', expand=True, padx=5, pady=5)

        for row_i, row in enumerate(buttons):
            for col_i, btn_text in enumerate(row):
                color = '#ff9f0a' if btn_text in '/*-+=' else '#333'
                btn = tk.Button(
                    frame, text=btn_text, font=('Arial', 18),
                    bg=color, fg='white', relief='flat',
                    command=lambda t=btn_text: self._press(t))
                btn.grid(row=row_i, column=col_i,
                        padx=2, pady=2, sticky='nsew')
                frame.grid_columnconfigure(col_i, weight=1)
                frame.grid_rowconfigure(row_i, weight=1)

    def _press(self, key):
        if key == 'C':
            self.expr = ''
        elif key == '⊞':
            self.expr = self.expr[:-1]
        elif key == '=':
            try:
                self.expr = str(eval(self.expr))
            except:
                self.expr = 'Ошибка'
        else:
            self.expr += key
        self.display.config(text=self.expr or '0')

root = tk.Tk()
Calculator(root)
root.mainloop()
```

 **Задание 7:** Создай GUI-приложение «Конвертер единиц»: выпадающий список с типами конвертации (км↔мили, кг↔фунты, °C↔°F), поле ввода, кнопка «Перевести», поле с результатом.

Глава 30: Упаковка и публикация проекта

30.1 Структура профессионального проекта

```
my_project/
├── my_package/           # основной пакет
│   ├── __init__.py      # делает папку пакетом
│   ├── core.py          # основная логика
│   └── utils.py          # вспомогательные функции
├── tests/               # тесты
│   ├── __init__.py
│   └── test_core.py
├── docs/                # документация
├── .env                 # секреты (не заливать в git!)
├── .gitignore           # что игнорировать в git
├── requirements.txt      # зависимости
├── README.md            # описание проекта
└── main.py              # точка входа
```

30.2 Виртуальное окружение

```
# Создать виртуальное окружение
python -m venv venv

# Активировать (Windows)
venv\Scripts\activate

# Активировать (Mac/Linux)
source venv/bin/activate

# Установить зависимости
pip install -r requirements.txt

# Сохранить зависимости
pip freeze > requirements.txt

# Деактивировать
deactivate
```

30.3 Git — контроль версий

```
# Инициализация репозитория
git init

# Добавить файлы
git add .
git add specific_file.py

# Сохранить изменения
git commit -m 'feat: добавить функцию авторизации'

# Посмотреть историю
git log --oneline

# Создать ветку для новой функции
git checkout -b feature/new-api

# Слить ветку в main
git checkout main
git merge feature/new-api

# Отправить на GitHub
git remote add origin https://github.com/user/repo.git
git push origin main
```

30.4 .gitignore — что не заливать

```
# Файл .gitignore
__pycache__/
*.pyc
*.pyo
.env
venv/
.venv/
*.db
*.sqlite3
.DS_Store
*.log
dist/
build/
*.egg-info/
```

□ **README.md — лицо проекта:** Хороший README содержит: краткое описание, скриншот, инструкцию по установке, примеры использования, список функций и лицензию. Оформляй на русском и английском.

□ Итоговый проект: Погодный дашборд

Объединяем веб-запросы, Pandas, Matplotlib и Flask в один реальный проект — веб-дашборд с графиком погоды.

```
# app.py — Flask + API погоды + Matplotlib
from flask import Flask, render_template, request
import requests, io, base64
import matplotlib
matplotlib.use('Agg') # без GUI (для сервера)
import matplotlib.pyplot as plt

app = Flask(__name__)

def get_forecast(city):
    url = f'https://wttr.in/{city}?format=j1'
    r = requests.get(url, timeout=8)
    r.raise_for_status()
    data = r.json()
    days = []
    for day in data['weather']:
        days.append({
            'date': day['date'],
            'max_c': int(day['maxtempC']),
            'min_c': int(day['mintempC']),
            'desc': day['hourly'][4]['weatherDesc'][0]['value']
        })
    return days

def make_chart(forecast):
    dates = [d['date'][5:] for d in forecast] # MM-DD
    maxes = [d['max_c'] for d in forecast]
    mins = [d['min_c'] for d in forecast]

    fig, ax = plt.subplots(figsize=(8, 3.5))
    ax.plot(dates, maxes, 'o-', color='tomato', label='Макс')
    ax.plot(dates, mins, 's--', color='steelblue', label='Мин')
    ax.fill_between(dates, mins, maxes, alpha=0.1, color='orange')
    ax.set_title('Прогноз температуры', fontsize=13)
    ax.set_ylabel('°C')
    ax.legend(); ax.grid(alpha=0.3)
    plt.tight_layout()

    buf = io.BytesIO()
    plt.savefig(buf, format='png', dpi=120)
    buf.seek(0)
    chart_b64 = base64.b64encode(buf.read()).decode()
    plt.close()
    return chart_b64

@app.route('/', methods=['GET', 'POST'])
def index():
    chart = forecast = error = None
    city = ''
    if request.method == 'POST':
        city = request.form.get('city', '').strip()
        try:
            forecast = get_forecast(city)
            chart = make_chart(forecast)
        except Exception as e:
            error = f'Ошибка: {e}'
```

```

        return render_template('weather.html',
                                chart=chart, forecast=forecast,
                                city=city, error=error)

if __name__ == '__main__':
    app.run(debug=True)

<!-- templates/weather.html -->
<!DOCTYPE html>
<html lang='ru'>
<head>
    <meta charset='UTF-8'>
    <title>Погода</title>
    <style>
        body { font-family: Arial; max-width: 700px; margin: 40px auto;
padding: 0 20px; }
        input { padding: 8px; font-size: 16px; width: 300px; border-radius:
6px; border: 1px solid #ccc; }
        button { padding: 8px 20px; background: steelblue; color: white;
border: none;
                border-radius: 6px; font-size: 16px; cursor: pointer; }
        .error { color: tomato; }
        table { width: 100%; border-collapse: collapse; margin-top: 20px; }
        th, td { padding: 8px 12px; border: 1px solid #ddd; text-align: center;
}
        th { background: steelblue; color: white; }
        tr:nth-child(even) { background: #f5f5f5; }
    </style>
</head>
<body>
    <h1>Погодный дашборд</h1>
    <form method='post'>
        <input name='city' placeholder='Введи город...' value='{{ city }}'>
        <button type='submit'>Показать</button>
    </form>
    {% if error %}<p class='error'>{{ error }}</p>{% endif %}
    {% if chart %}
        <img src='data:image/png;base64,{{ chart }}' style='width:100%;margin-
top:20px'>
        <table>
            <tr><th>Дата</th><th>Макс °C</th><th>Мин
°C</th><th>Описание</th></tr>
            {% for d in forecast %}
                <tr><td>{{ d.date }}</td><td>{{ d.max_c }}</td><td>{{ d.min_c
}}</td><td>{{ d.desc }}</td></tr>
            {% endfor %}
        </table>
    {% endif %}
</body>
</html>

```

□ **Что использовано:** requests (API) • Matplotlib (график) • Flask (веб) • Jinja2 (шаблоны) • base64 (картинка в HTML) • обработка ошибок

 **Задание 8:** Расширь дашборд: добавь историю поиска городов (сохраняй в session), кнопку «Сравнить два города» — два графика рядом, и возможность скачать CSV с прогнозом.

□ Финальное слово и портфолио

Ты прошёл весь курс!

За 4 части ты освоил Python от нуля до реальных проектов. Теперь главное — практика. Вот список проектов для портфолио:

Проект	Что нужно	Уровень
Чат-бот с ИИ	OpenAI API + aiogram	★★★
Парсер вакансий	requests + BS4 + Pandas	★★☆
REST API для блога	Flask/FastAPI + SQLite	★★☆
Анализ акций	yfinance + Pandas + Matplotlib	★★★
Telegram-менеджер задач	aiogram + SQLite + FSM	★★★
Конвертер документов	python-docx + openpyxl	★★☆
Игра на Pygame	Pygame	★★★
Автоматизация браузера	Selenium	★★★
CLI-утилита	argparse + rich	★☆☆
Дашборд с данными	Flask + Pandas + Matplotlib	★★★

Следующий шаг: публикуй свои проекты

- Создай аккаунт на GitHub и загружай каждый проект
- Пиши README на русском и английском
- Участвуй в open-source: исправляй баги в чужих проектах
- Решай задачи на LeetCode — хотя бы 1 задачу в день
- Найди ментора или вступи в Python-сообщество (t.me/ru_python)

□ **Главный совет:** Лучший способ выучить программирование — писать код каждый день. Даже 30 минут в день за год дадут огромный результат. Не бойся ошибок — они часть обучения!